Product Case Study: Improving Cost Transparency in Emergent

By - Kartikey Agarwal (IIT Delhi'24 | Fintech & Agentic AI Product Builder & Technical Executer | sbikartikey0911@gmail.com)

Problem

Unpredictable credit consumption reduces user trust and discourages iterative experimentation, negatively impacting retention and expansion revenue.

Users often begin a build without understanding the computational scope of their request. When the system consumes credits unpredictably or fails after high credit burn, users perceive the platform as unreliable or expensive. This reduces experimentation and increases early churn.

Assumptions

• **~2%** of monthly churn is driven by cost confusion during early usage.
• Average credit usage per build session ≈ **200 credits** (150 build + 50 deployment).
• **~30%** of build attempts fail after high credit consumption due to agent inefficiencies or prompt ambiguity.
• Current CAC:LTV ratio ≈ **1:3** with strong LTV upside if retention improves.

Objective

Increase perceived cost transparency and reduce credit anxiety while maintaining revenue growth and high ltv.

Non-goals

This proposal does **not** aim to:

• Introduce unlimited free usage or significantly lower subscription pricing
• Improve model accuracy or generation speed
• Replace the current credit-based pricing model with flat-rate pricing
• Redesign the core agent architecture

Target Users/ Personas

Non-technical founder-

- **Goal:** Launch an MVP quickly to validate product ideas with minimal engineering investment.

- **Pain Points**
    - Difficulty translating product ideas into structured technical requirements
    - Unpredictable development costs when iterating
    - Fear of appearing technically incompetent when presenting demos to investors
- **Behavioral Insights**
    - Optimizes for visible progress rather than architectural correctness
    - Often assumes feature requests are simple when underlying complexity is high
    - Values rapid demo creation more than system robustness
- **Success Metrics**
    - Time to MVP launch
    - Number of iterations before demo-ready product
    - Early user validation

Full-stack engineer-

- **Goal:** Accelerate development by automating repetitive tasks while retaining control over architecture and code quality.
- **Pain Points**
    - Time spent writing boilerplate code
    - Debugging poorly structured generated code
    - Risk of regression when modifying AI-generated components
- **Behavioral Insights**
    - Distrusts generated code until it proves reliable
    - Prioritizes debuggability and modular structure
    - Avoids tools that create vendor lock-in
- **Success Metrics**
    - Developer velocity
    - Debugging time
    - Deployment frequency

Business Persona (Growth/ Monetization Lead)-

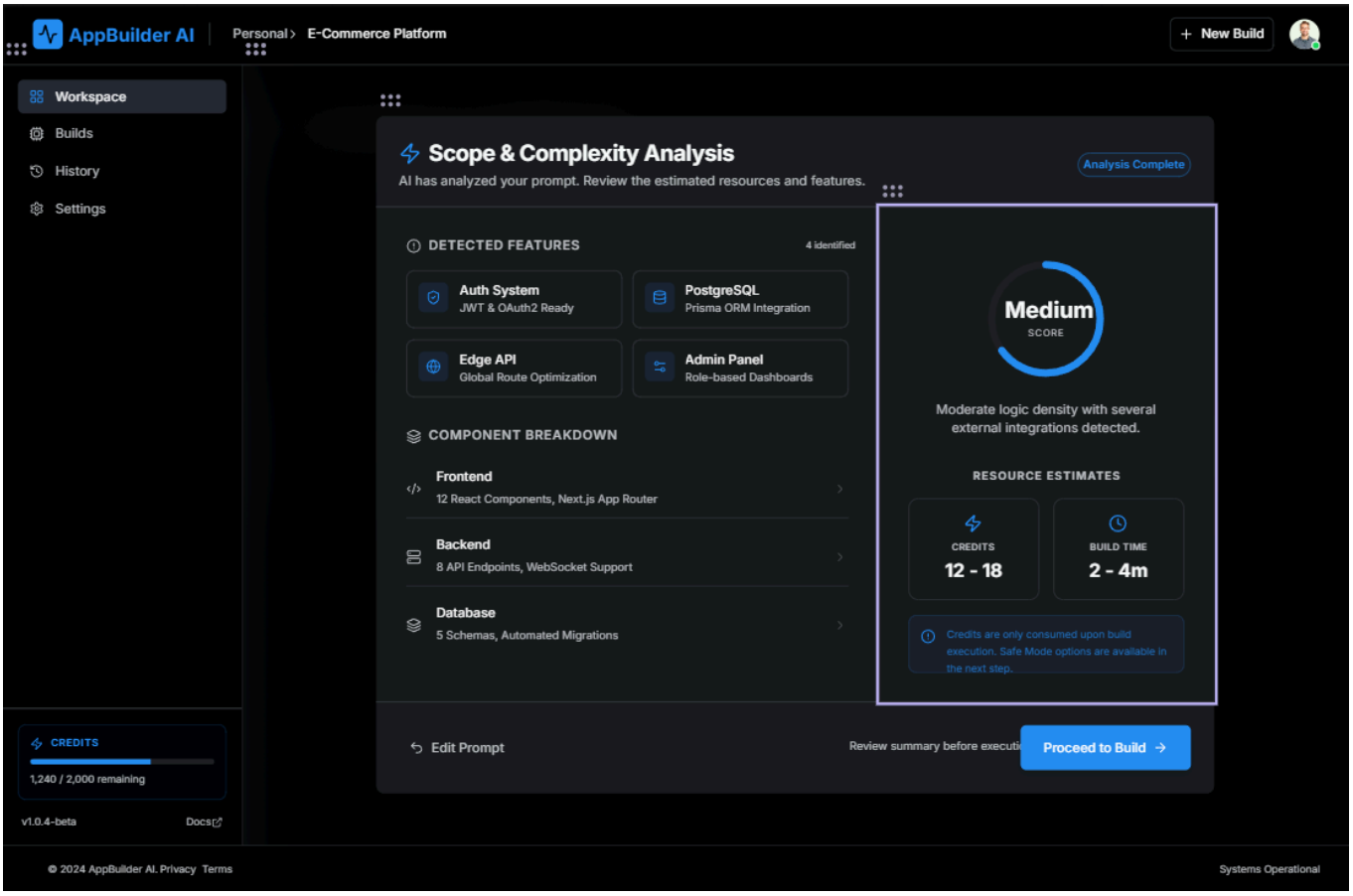
- **Goal:** Increase paid conversion and expansion revenue while maintaining long-term user trust.
- **Key Concerns**
    - Early churn after unexpected credit usage
    - Infrastructure costs from failed builds
    - Abuse of free credits without meaningful engagement
- **Success Metrics**
    - Paid conversion rate
    - Credit utilization per user
    - Month-1 retention
    - Customer lifetime value (LTV)
- **Trust Breaker:** Users perceiving pricing as opaque or unfair.

Principles:

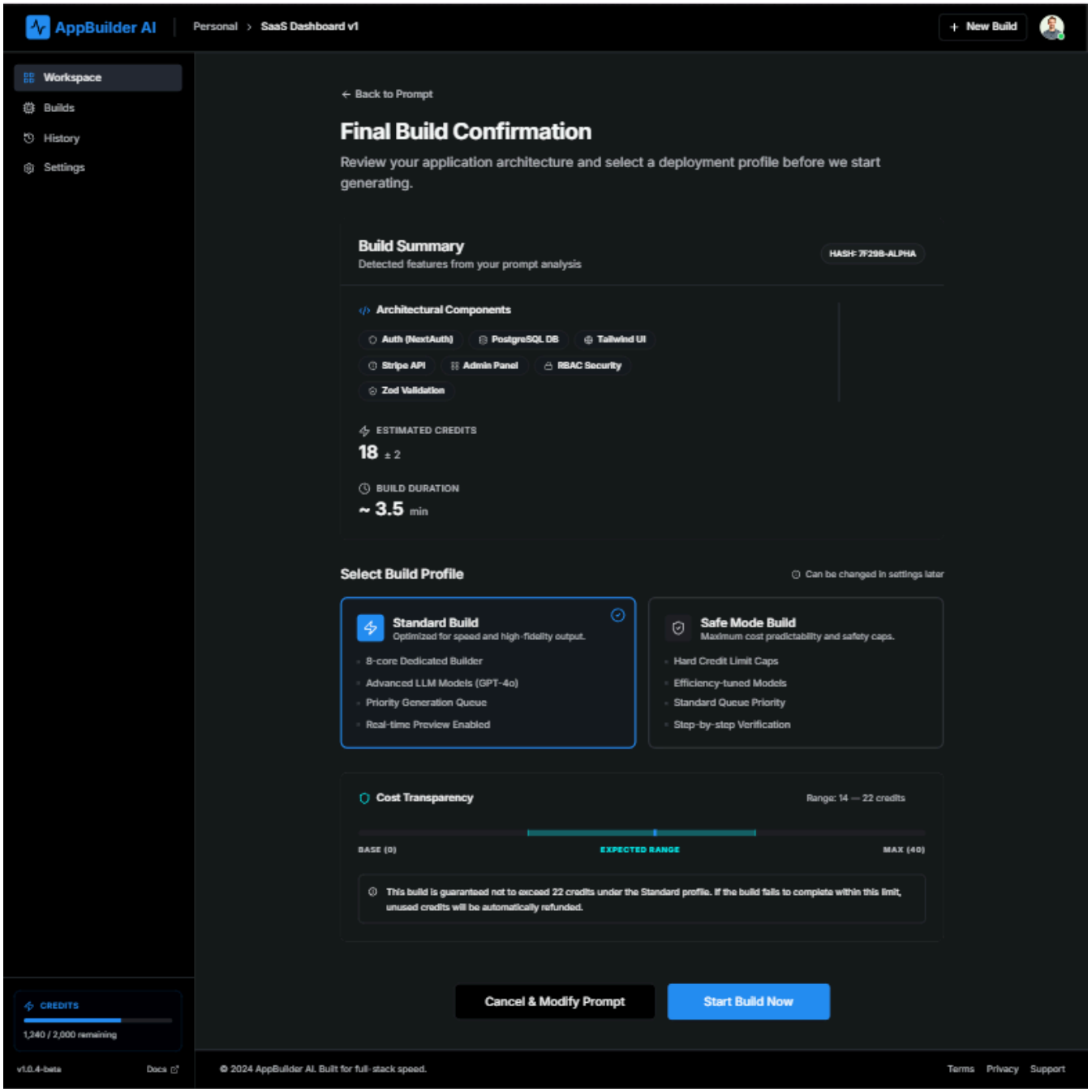
1. **Predictability over Surprise**
   Users should understand expected cost and scope before execution.
2. **Transparency over Abstraction**
   The system should expose meaningful information about generation progress, cost usage, and reasoning.
3. **Control over Automation**
   Users must be able to inspect, modify, and override system-generated outputs.
4. **Progressive Commitment**
   Users should evaluate and refine plans before committing compute resources or credits.
5. **Feedback over Silence**
   The system must continuously communicate progress, errors, and system state.
6. **Safe Iteration**
   Users should be able to experiment without fear of excessive cost or irreversible changes.
7. **Scope Clarity before Execution**
   The system must model and communicate detected feature scope before generation begins.

Features:

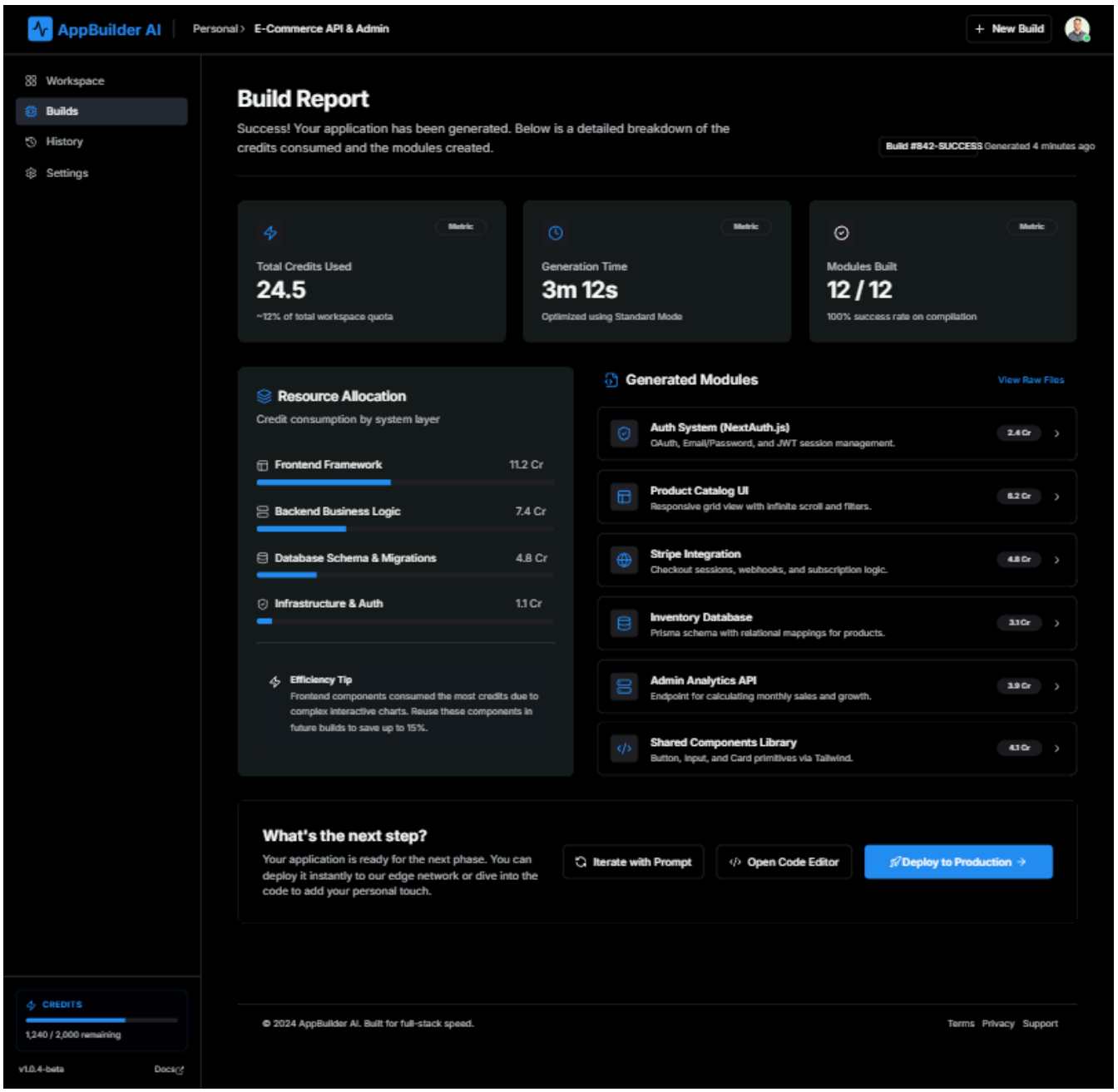
1. Scope and complexity estimator before build: A pre-build modal/page that analyzes the user's prompt and *predicts the feature set, complexity score, and estimated credit range* before any code runs.



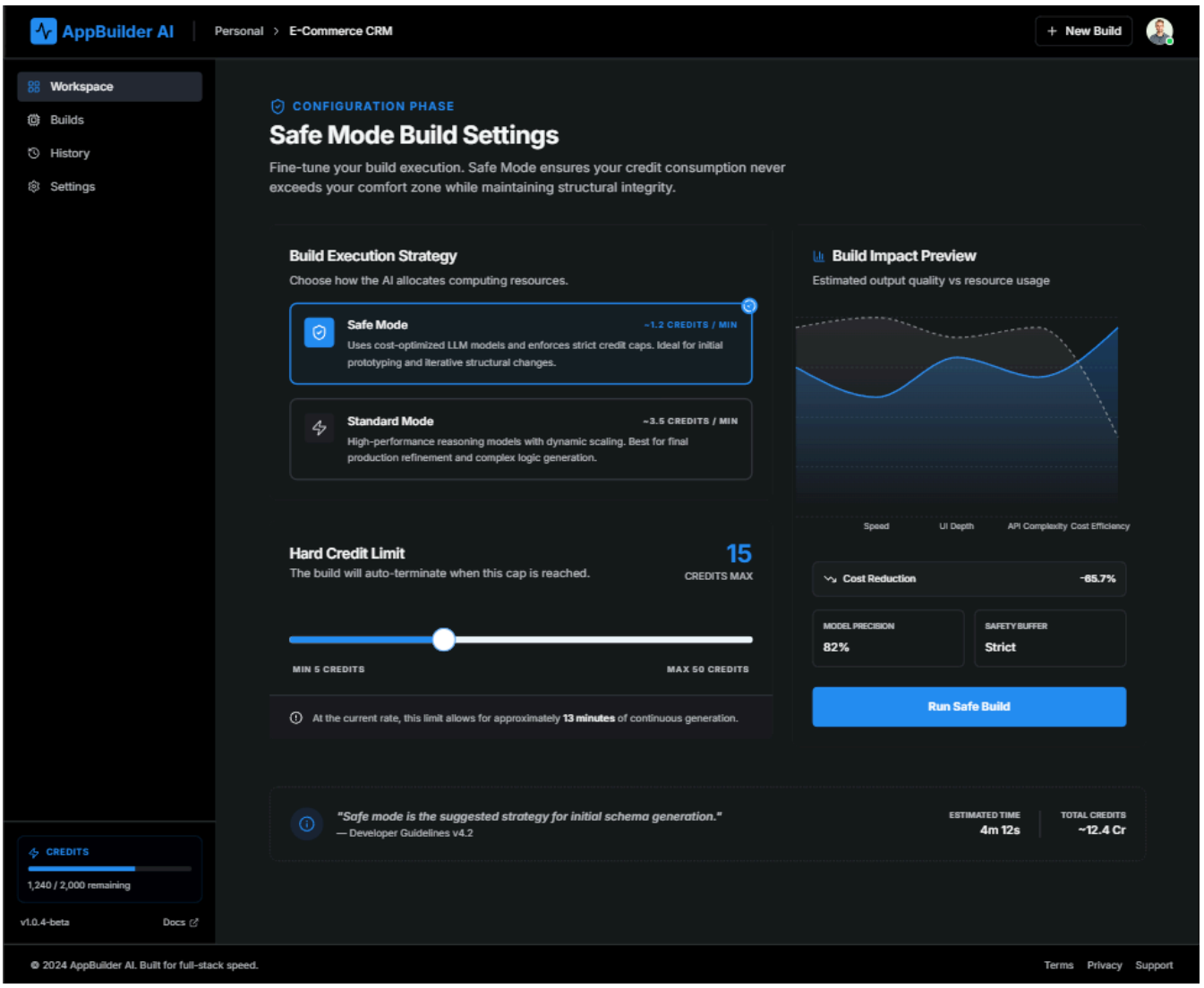
2. Real time Credit burn meter: A live indicator (progress bar + numeric readout) showing how many credits have been used and how many remain *during the build process*.
3. Pre-execution confirmation screen



4. Post build breakdown report: After build, show a detailed breakdown of - Credits spent per component, Time spend, Code modules created, Next recommended action based on the user behaviour.



5. “Safe mode” build option with capped burn: Alternative build profiles where users can set a max credit limit and system prioritizes simpler models and progressive outputs to stay under budget.



6. Build versionaning and easy rollback: Automatic version tracking of builds with a rollback option to resvert to a previous working state.

AppBuilder AI | Personal > E-Commerce Starter | + New Build

- Workspace
- Builds
- History
- Settings

Build History

Track, preview, and restore previous versions of your application infrastructure.

TOTAL BUILDS 42 | TOTAL SPENT 842

Search version or build description... | All | Successful | Failed | Filters

v1.2.4 Completed
Added authentication middleware and Stripe integration webhooks.
Oct 24, 2024 • 14:20 | 3m 42s
COST: 18 Credits | ENGINE: GPT-4o (Standard) | EFFICIENCY
Preview Build | Rollback | Duplicate

v1.2.3 Completed
Updated landing page hero section with new copy and primary CTA.
Oct 23, 2024 • 09:12 | 2m 15s
COST: 12 Credits | ENGINE: GPT-4o (Standard) | EFFICIENCY
Preview Build | Rollback | Duplicate

v1.2.2 Failed
Implemented Redis caching layer for dashboard analytics.
Oct 22, 2024 • 18:55 | 0m 45s
COST: 4 Credits | ENGINE: GPT-4o (Standard) | EFFICIENCY
Preview Build | Rollback | Duplicate

v1.2.1 Completed
Initial core infrastructure build with PostgreSQL and Express.js.
Oct 21, 2024 • 11:30 | 5m 10s
COST: 22 Credits | ENGINE: GPT-4o (Standard) | EFFICIENCY
Preview Build | Rollback | Duplicate

v1.1.9 Completed
Patch for navigation mobile responsiveness issues.
Oct 19, 2024 • 15:45 | 3m 02s
COST: 15 Credits | ENGINE: GPT-4o (Standard) | EFFICIENCY
Preview Build | Rollback | Duplicate

Load previous builds >

Showing 5 of 42 builds across this project's history.

About Rollbacks

Rolling back to a previous version will re-deploy the exact infrastructure, database schema, and UI components from that point in time. Note that rolling back consumes 5 credits for re-verification and state sync. Your current active build will be saved as a new version.

Version Control Documentation

CREDITS
1,240 / 2,000 remaining

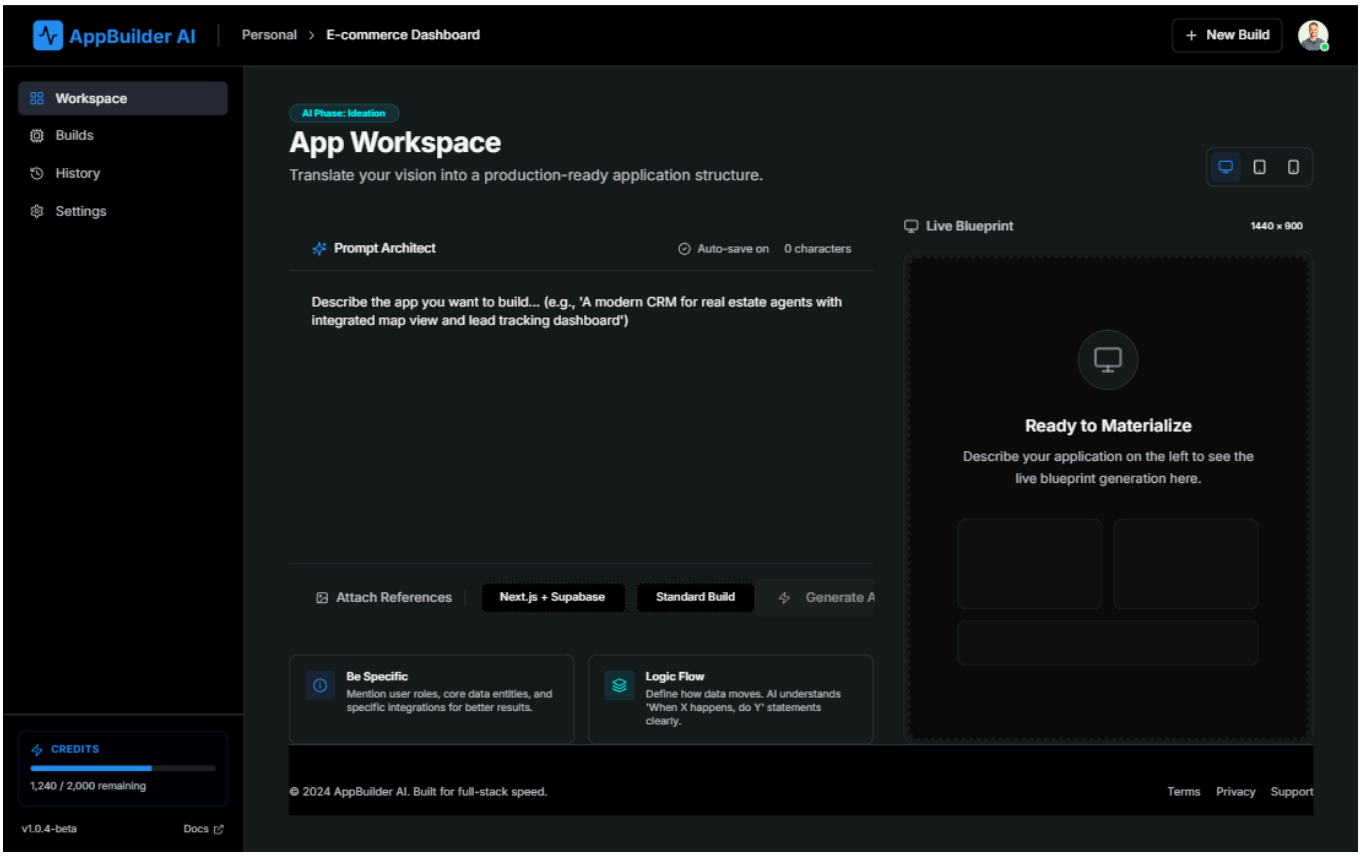
v1.0.4-beta | Docs

© 2024 AppBuilder AI. Built for full-stack speed.

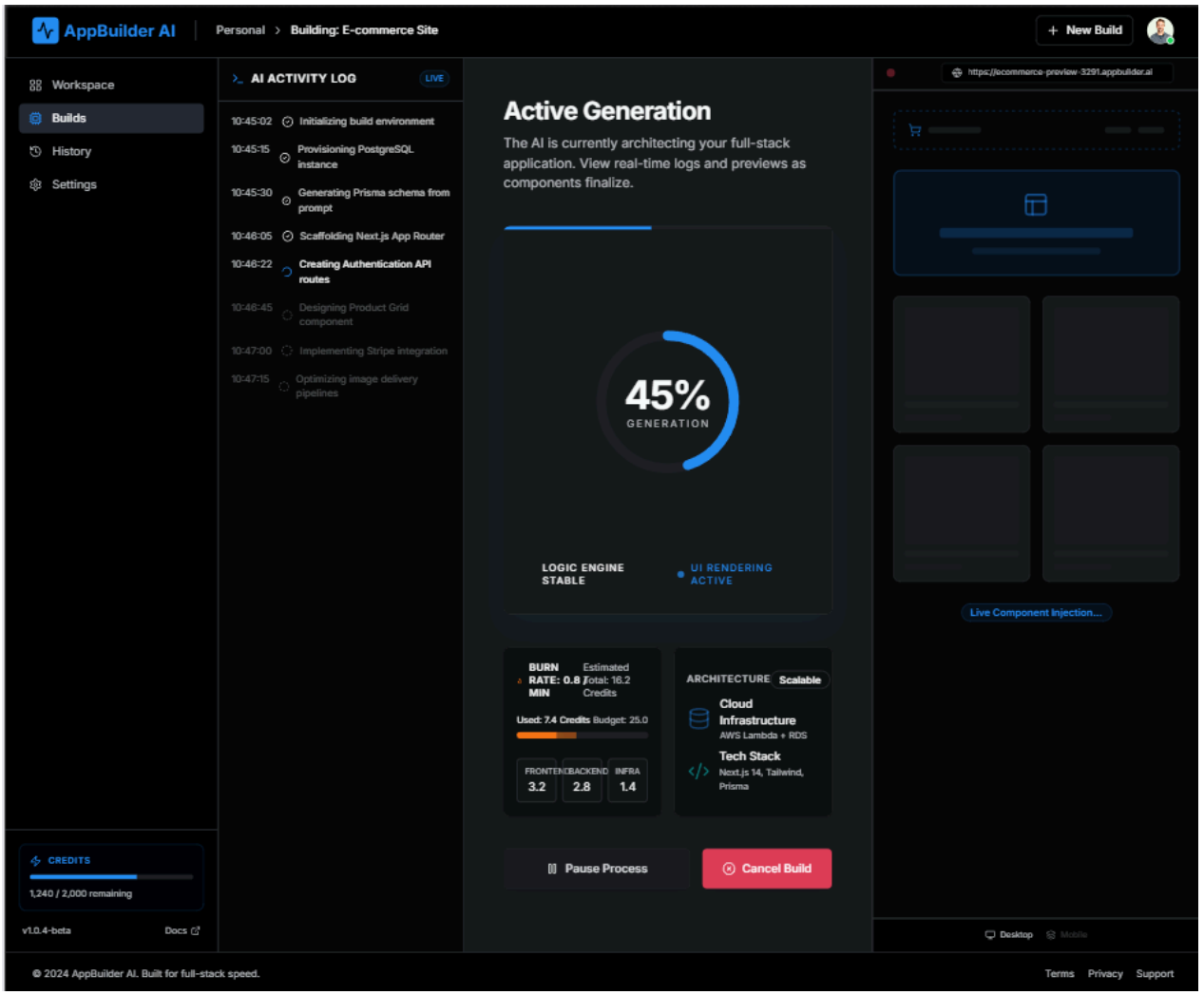
Terms | Privacy | Support

More Wireframes & Screens:

1. Workspace & Onboarding page



2. Building app



Metrics:

The objective of this solution is to reduce cost anxiety, improve build success, and increase user trust in the system while maintaining healthy monetization.

Primary Metric

Early User Churn Due to Cost Frustration

- Definition: Percentage of new users who stop using the platform after their first build due to perceived high or unpredictable credit usage.
- Why this matters: The core problem being addressed is expectation mismatch around cost and build complexity. Reducing early churn indicates improved trust and a better first-time experience.
- Measurement window: First 7 days after the user's first build.
- Success signal: Lower drop-off after the first build attempt.

Secondary Metrics

Credits per Successful Build

- Definition: Average number of credits consumed per completed build session.
- Purpose: Measures efficiency of generation and whether the platform is helping users achieve successful outputs without excessive retries.
- Desired trend: Stable or slightly reduced credit usage with higher success rates.

Iteration Frequency

- Definition: Average number of build iterations per project.
- Purpose: Measures whether users feel safe experimenting and refining their application.
- Desired trend: Moderate increase, indicating healthier iteration behavior.

Build Completion Rate

- Definition: Percentage of builds that successfully complete without failure or user abandonment.
- Purpose: Indicates improved reliability and reduced agent inefficiencies.
- Desired trend: Increase in successful build completion.

Counter Metric

Revenue per Active User

- Definition: Average revenue generated per active user within a billing cycle.
- Purpose: Ensures transparency improvements do not unintentionally reduce monetization.
- Desired trend: Stable or gradually increasing as user retention improves.

Risks & Tradeoffs

Risk 1: Reduced Credit Consumption

If users understand costs before execution, they may optimize prompts more carefully and reduce unnecessary builds.

- Impact: Short-term reduction in credit consumption per user.
- Tradeoff: Lower revenue per build but potentially higher retention and long-term monetization.
- Mitigation: Encourage iterative development workflows and highlight new feature suggestions after successful builds.

Risk 2: Reduced Experimentation

Users may become overly cautious if cost estimates appear high.

- Impact: Users may avoid testing larger ideas or complex applications.
- Tradeoff: Lower iteration frequency and slower product exploration.
- Mitigation: Introduce safe build modes and staged execution options that allow experimentation within predictable credit limits.

Risk 3: Estimation Inaccuracy

Scope estimation may occasionally under-estimate or over-estimate the real cost of generation.

- Impact: Incorrect estimates may reduce trust if actual costs deviate significantly.
- Tradeoff: Over-estimation discourages builds; under-estimation damages credibility.
- Mitigation: Display estimated credit ranges and confidence scores rather than fixed values.

Risk 4: Increased System Complexity

Adding planning and estimation layers introduces additional system overhead.

- Impact: Slightly longer time before build execution.
- Tradeoff: Small increase in latency but improved predictability and decision clarity.
- Mitigation: Optimize planning phase to remain lightweight and fast.

Appendix A - Product Research Notes

Product Overview

Emergent focuses on generating **production-ready full-stack applications** (web + mobile) from natural language prompts.

Key characteristics observed:

• Agent-based workflow where specialized agents handle planning, generation, and deployment
• Generates full stack output (frontend, backend, database)
• Built-in deployment pipeline with minimal configuration
• Code ownership through GitHub export
• Designed for both non-technical builders and developers

Competitor Comparison:

a. **Replit:** Strong collaborative coding environment and rapid prototyping workflow.
b. **Cursor:** AI coding assistant optimized for developers writing and learning code.

c. **Lovable:** Focus on aesthetic, design-first frontend generation.
d. **Base44:** Emphasis on data visibility and security.
e. **Bolt.new:** Very fast for early prototypes and experimentation.

Observation: Most competitors optimize for **either developers or prototypes**, while Emergent attempts **production-ready generation**.

User flow and Early friction mapping

1. Landing & Onboarding

User action: User visits emergent.sh, explores features and pricing, signs up via email or GitHub.

System behavior: User enters a dashboard workspace with a prompt field (default: “Build me a…”).

Early friction:

- Free tier credits extremely limited (≈5 credits)
- Cost expectations unclear early in journey
- Occasional agent initialization errors

2. Prompt Creation

User action: User describes the app idea in natural language and optionally adds references or advanced settings.

System behavior: Agents analyze prompt and create an internal plan for app architecture.

Early friction:

- Prompt vagueness leads to generic output
- Complex prompts confuse agents
- Users struggle to translate product ideas into structured requirements

3. AI Build Generation

User action: User watches agents generate the application while preview and code appear.

System behavior: Platform generates frontend, backend, and database simultaneously.

Early friction:

- Generation delays or agent stalls
- UI output not matching user mental model
- Early design iterations often required

4. Code Inspection

User action: User reviews generated code in the IDE.

System behavior:
System organizes components, APIs, and database schemas.

Early friction:

- Debugging difficult for non-technical users
- AI-generated code sometimes hard to modify cleanly

5. Deployment

User action: User deploys application to a live URL.

System behavior: Platform handles hosting, SSL, and infrastructure.

Early friction:

- Deployment consumes significant credits
- Custom domain setup not always straightforward

6. Iteration

User action: User refines application via prompt updates.

System behavior: Agents regenerate affected parts and redeploy.

Early friction:

- Regression bugs when modifying existing features
- Iteration repeatedly consumes credits

Problems to solve

1. High-Cost and Unpredictable Credit Consumption: Users frequently report that credits burn too fast, making it expensive for anything beyond simple, single-feature apps.
2. Reliability, Stability, and Agent "Hallucinations": The AI agents, while powerful, often struggle with complex, multi-layered applications, resulting in broken, inconsistent code.
3. Lack of Production-Ready Transparency and Security: While marketed as "production-ready," the platform lacks the maturity needed for commercial, business-critical applications.
4. Users don’t know how to write prompts that get good results.

Key insight from research: Most user frustration originates from **expectation mismatch between prompt complexity and actual system cost.**